an `Authorization: Bearer` header just as you've done in previous chapters. You'll see how to validate an access token in your API in section 7.4.

7.3.1 Redirect URIs for different types of client

The choice of redirect URI is an important security consideration for a client. For public clients that don't authenticate to the AS, the redirect URI is the only measure by which the AS can be assured that the authorization code is sent to the right client. If the redirect URI is vulnerable to interception, then an attacker may steal authorization codes.

For a traditional web application, it's simple to create a dedicated endpoint to use for the redirect URI to receive the authorization code. For a single-page app, the redirect URI should be the URI of the app from which client-side JavaScript can then extract the authorization code and make a CORS request to the token endpoint.

For mobile applications, there are two primary options:

- The application can register a private-use URI scheme with the mobile operating system, such as `myapp://callback`. When the AS redirects to `myapp://callback?code=…` in the system web browser, the operating system will launch the native app and pass it the callback URI. The native application can then extract the authorization code from this URI and call the token endpoint.